Politechnika Lubelska
Wydział Elektrotechniki i Informatyki
Katedra Elektroniki i Technik Informacyjnych

Systemy wizyjne

Prowadzący laboratorium: mgr inż. Marek Kida

Ćwiczenie laboratoryjne nr 4

Temat: Detekcja ruchu za pomocą usuwania tła

Piotr Bednarski

Lublin, 08.12.2025 r.

Ćwiczenie 1. Usuwanie tła

Stwórz nowy folder, w którym znajdować się będą pliki. Struktura plików ma wyglądać następująco:

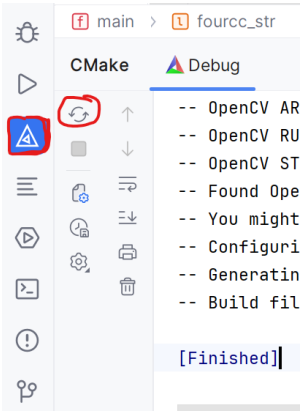
```
/home/student/Documents/
  - number_albumu/
      - L4/
          - latex/
          - code/
```

Do folderu `latex` wklej rozpakowany szablon sprawozdania.

Stwórz nowy projekt w programie CLion w języku C++ (*File -> New -> Project -> C++ Executable*) w folderze `/home/student/CLionProjects/numer_albumu/L4/code`. Zamień zawartość pliku `CMakeLists.txt` na następującą:

```
cmake_minimum_required(VERSION 3.30)
project(NAZWA_PROJEKTU)
find_package(OpenCV REQUIRED)
include_directories(${OpenCV_INCLUDE_DIRS})
add_executable(${PROJECT_NAME} main.cpp)
target_link_libraries(${PROJECT_NAME} ${OpenCV_LIBS})
```

Zmień nazwę projektu na nazwę folderu, w którym znajduje się projekt. Odśwież zawartość projektu CMake w panelu na lewym dolnym roku ekranu. Dzięku temu edytor C++ będzie znajdował bibliotekę OpenCV.



Rysunek 1: Przeładowanie projektu CMake

Wprowadź do programu CLion poniższy kod. Po wklejeniu kodu sformatuj go za pomocą `ALT + SHIFT + F`. Do folderu `cmake-build-debug` wprowadź plik `highway.mp4` poprzez naciśnięcie prawego przycisku myszy i wybranie *Save image as...*.

```
#include <opencv2/objdetect.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/videoio.hpp>
#include <opencv2/video/background_segm.hpp>
#include <opencv2/bgsegm.hpp>
#include <iostream>
#include <iomanip>

int main(int argc, char **argv) {
    cv::Mat frame_ref, frame_mask;

    auto capture = cv::VideoCapture("url_lub_sciezka");
    auto subtractor = cv::createBackgroundSubtractorMOG2(100, 100);

    while (capture.read(frame_ref)) {
        if (cv::waitKey(10) & 0xFF == 27) {
            break;
        }

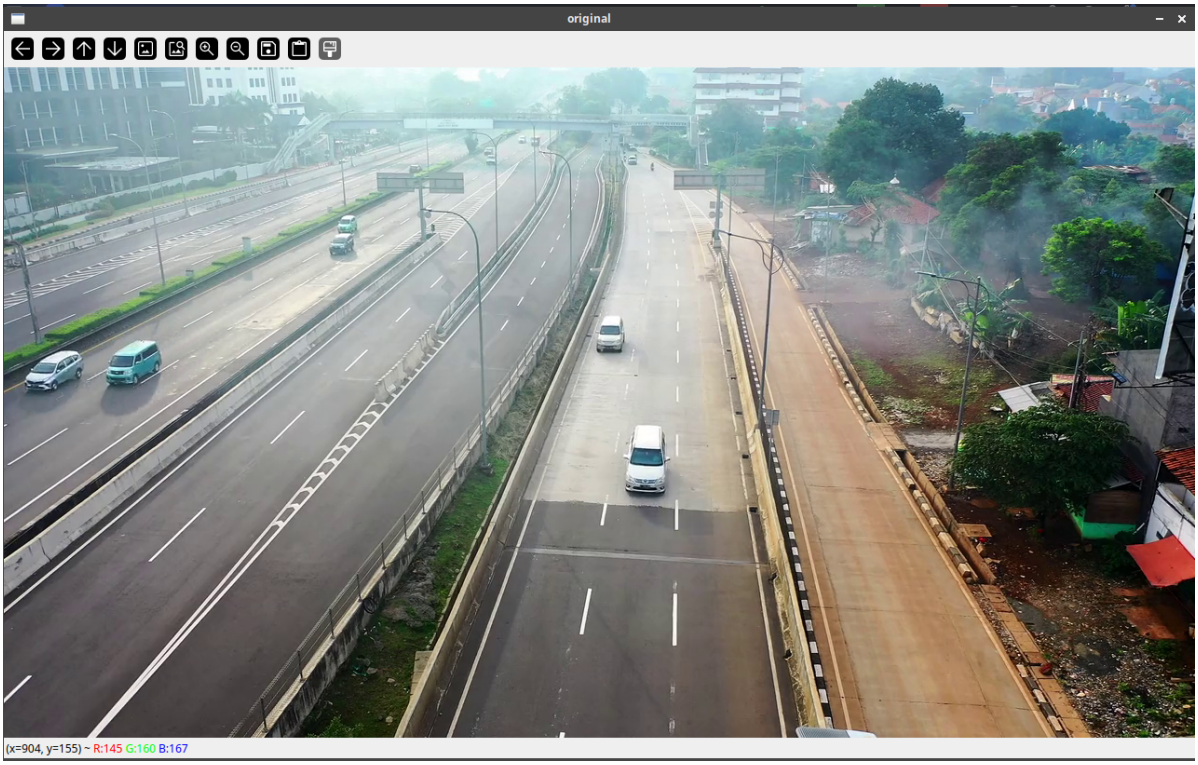
        subtractor->apply(frame_ref, frame_mask);

        cv::imshow("original", frame_ref);
        cv::imshow("mask", frame_mask);
    }

    capture.release();
    cv::destroyAllWindows();
    return 0;
}
```

W odpowiedzi zawrzyj następujące informacje i zdjęcia:

1. Wyświetl obie klatki wideo i przedstaw, które obiekty dobrze widoczne są w masce uzyskanej przez algorytm MOG2. Jakie niepożądane artefakty pojawiają się na obrazie.



Rysunek 2: Film oryginalny



Rysunek 3: Widok maski

Dobrze widoczne obiekty to poruszające się na autostradzie pojazdy. Za zbędne artefakty można uznać kontury drzew i budynków, zarysy latarni przy drodze a także (w zależności od zastosowania systemu wizyjnego) linie oddzielające pasy ruchu.

2. Jak wygląda ta maska bez wykrywania i usuwania cieni? Sprawdź parametry funkcji `cv::createBackgroundSubtractorMOG2()` w dokumentacji (najedź na funkcję i trzymając Ctrl wciśnij lewy przycisk), a następnie użyj domyślnych wartości oprócz wykrywania cieni.



Rysunek 4: Widok maski bez wykrywania cieni

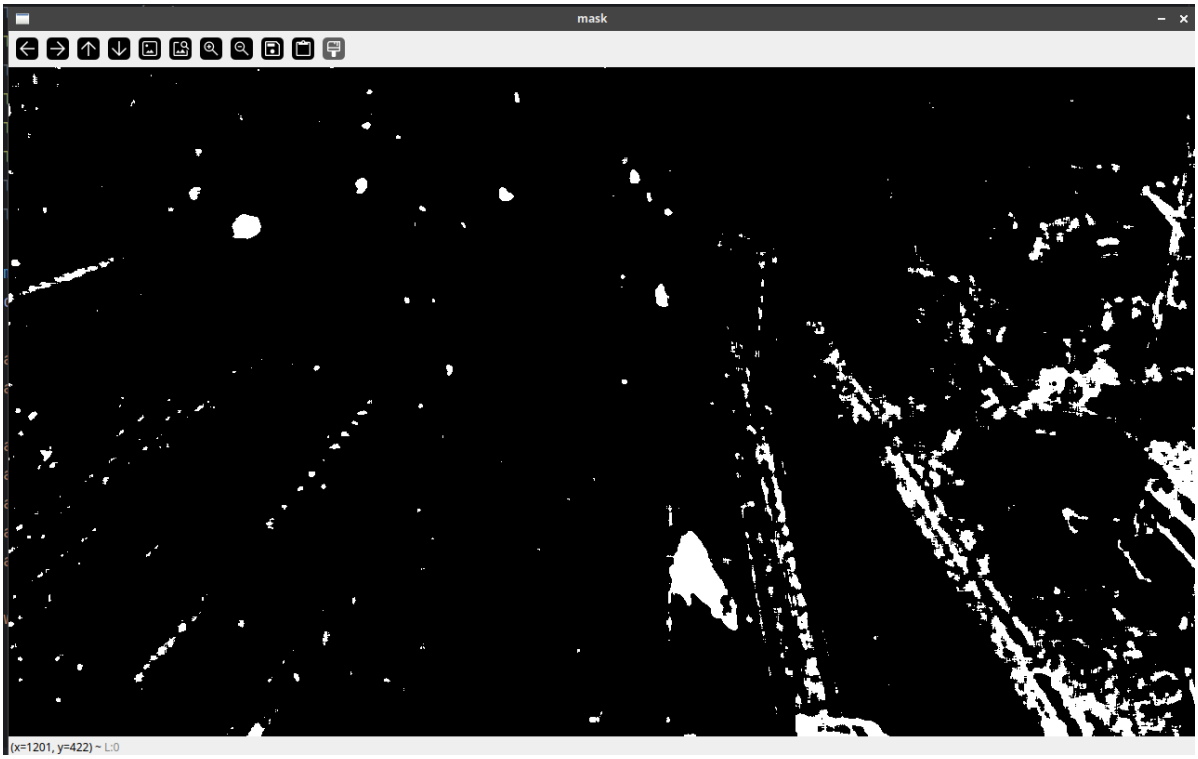
Ilość oraz wyraźność wyżej opisanych możliwie niepotrzebnych obiektów wzrosła, szczególnie w momencie filmu, gdzie następuje obrót kamery. Zaletą jest fakt, że algorytm działa szybciej.

3. Sprawdź zachowanie innych algorytmów usuwania tła. Aby je znaleźć, wpisz początek polecenia `cv::bgsegm::createBackground` i poczekaj na podpowiedź programu. Wykonaj po jednym zdjęciu dla każdego dostępnego algorytmu z dostępnych (maska + oryginał). Porównaj te algorytmy, opisując które z nich lepiej radzą sobie z cieniami, które lepiej rozpoznają pojazdy, które są bardziej zaszumione, które mają większe ubytki w sylwetkach wykrytych obiektów. Do kolejnych zadań wykorzystaj już poznany MOG2.



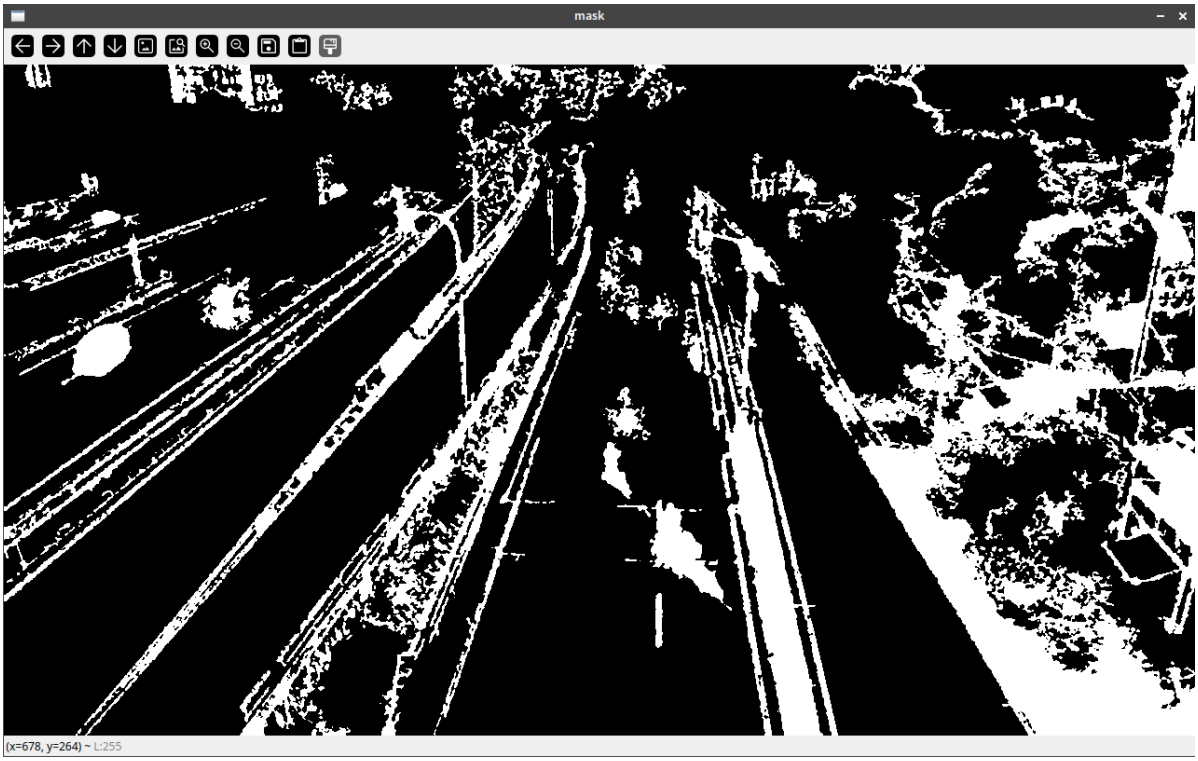
Rysunek 5: Maska algorytmu CNT

Szybki algorytm, ale znajduje się na nim zdeecydowanie za wiele artefaktów, tj. obiektów nie interesujących nas z punktu widzenia tego systemu wizyjnego.



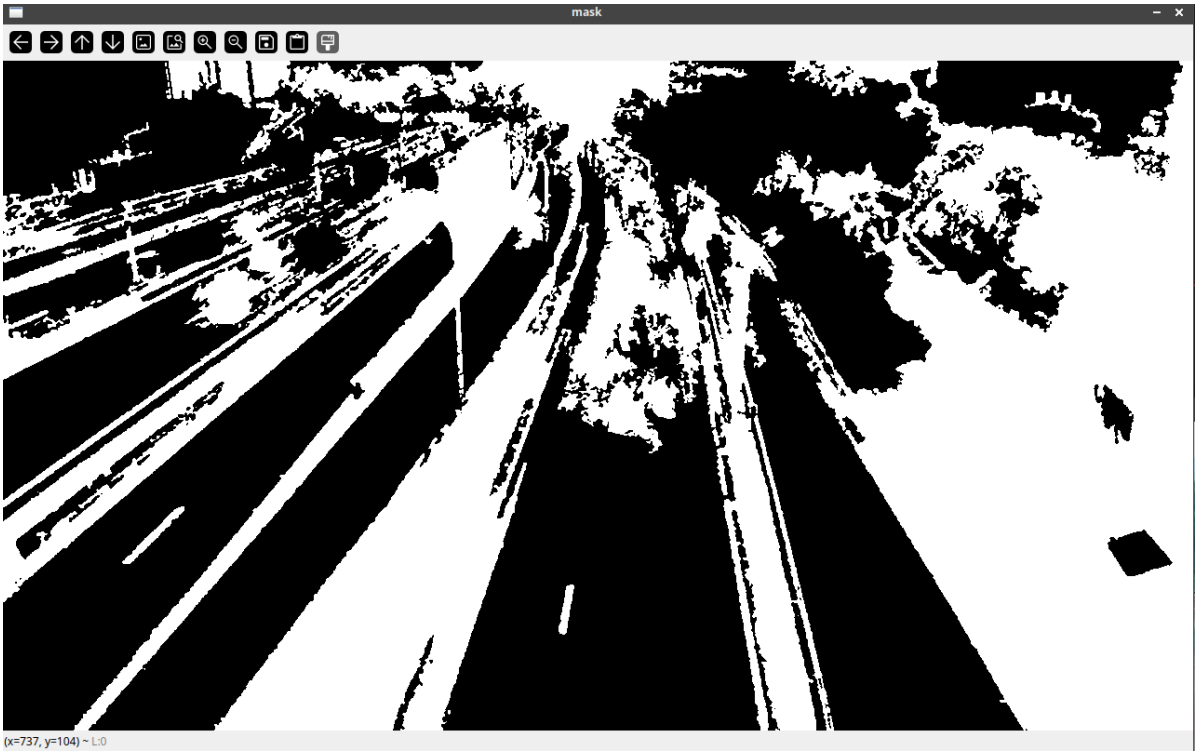
Rysunek 6: Maska algorytmu GMG

Bardzo powolny algorytm, generujący plamy, tj. zbiory punktów stykających się ze sobą. Niestety, artefakty też są plamami, zatem b. trudno rozróżnić je od interesujących nas obiektów. Alborytm zdecydowanie niezalecany do tego zadania.



Rysunek 7: Maska algorytmu GSOC

Algorytm nieco szybszy niż GMC, również generuje plamy, i również artefakty zaszumiają obraz w zbyt dużym stopniu.



Rysunek 8: Maska algorytmu LSBP

Algorytm generujący zdecydowanie zbyt dużo szumu. Obiekty będące przedmiotem zainteresowań są niemal niewidoczne.



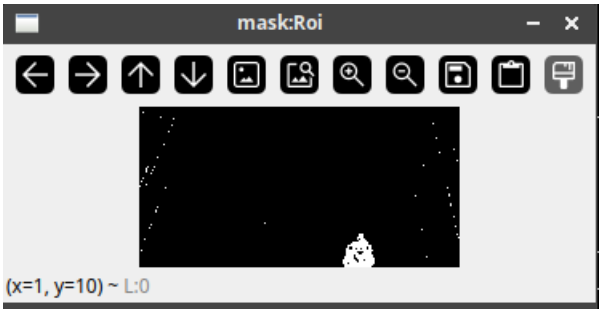
Rysunek 9: Maska algorytmu MOG

Algorytm szybki, obiekty będące przedmiotem zainteresowań systemu wizyjnego dobrze widoczne i łatwo rozróżnialne od artefaktów będących szumem. Nieco więcej artefaktów niż na MOG2.

4. Stwórz i wyświetl nową klatkę stanowiącą wycinek jezdni dla której wykrywane pojazdy. Dla tej klatki zastosuj algorytm usuwania tła zamiast całego obrazu. Aby zaznaczyć wybrany obszar zainteresować można użyć:

```
cv::Mat frame_roi = frame(cv::Rect(x, y, w, h));
```

Wymiary tego obszaru przechowaj w zmiennych, przydadzą się później.



Rysunek 10: Utworzony obszar zainteresowań ROI o wymiarach 200x100

Ćwiczenie 2. Znajdowanie i wyświetlanie konturów

Zmodyfikuj istniejący program, tak, aby wyświetlał kontury obiektów wyświetlanych przez maskę. W tym celu dodaj następujący kod:

```
std::vector<std::vector<cv::Point>> contours;
std::vector<cv::Vec4i> hierarchy;
cv::findContours(frame_in, contours, hierarchy, cv::RETR_TREE,
 ↪  cv::CHAIN_APPROX_SIMPLE);

for (size_t i = 0; i < contours.size(); i++) {
    double area = cv::contourArea(contours[i]);
    if (area > 100) {
       cv::drawContours(frame_in, contours, -1, cv::Scalar(255, 0, 0),
            2, cv::LINE_AA, hierarchy, 3, cv::Point(x, y));
    }
}
```

Wstaw go przed wyświetleniem obrazu. `drawContours()` ma za zadanie dodać obrysy na oryginalny, kolorowy obraz.

1. Jaki jest wpływ wartości wpisanej w warunku `area > 100` na wyświetlane kontury? Podaj kilka przykładów.

   Wartość x warunku area > x to ilość pikseli, od którego funkcja rysująca kontury uznaje, że w obszarze jest jakiś obiekt, którego kontury należy narysować. Zbyt mała wartość spowoduje konturowanie szumów. Zbyt duża wartość spowoduje nie wykrycie niektórych mniejszych obiektów (np. dla area > 800 obszar zainteresowań

zostanie zakonturowany dopiero, gdy znajdują się w nim samochód i dwa motocykle)

2. Jaki jest wpływ parametru `contourIdx`, którego domyślna wartość wynosi -1?

   Wpływ parametru contourIdx jest taki, że gdy jest ujemny, to rysowane są wszystkie kontury, a gdy jest dodatni, to rysowana jest kontura o indeksie contourIdx z wejściowego wektora kontur. Jeśli długość wektora wejściowego kontur jest mniejsza niż contourIdx, program wyrzuca wyjątek.

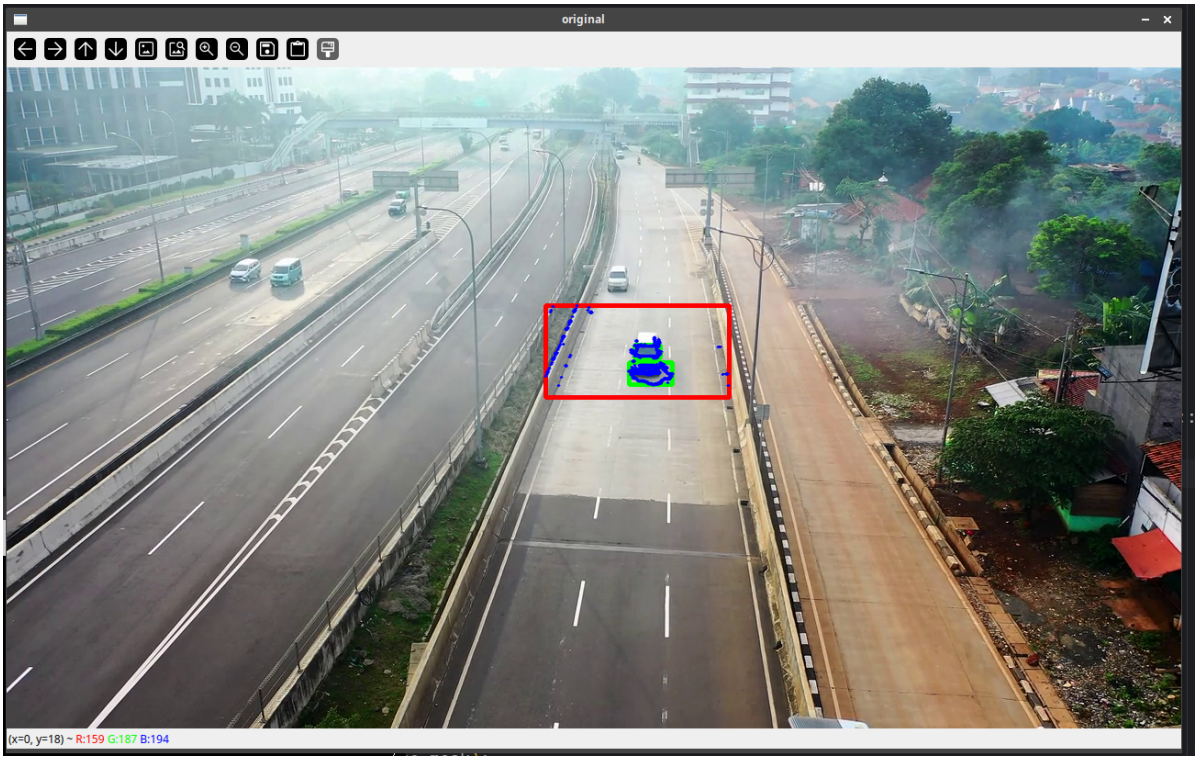
3. Napisz dlaczego zmienna zawierająca kontury to wektor wektorów punktów.

   Zmienna zawierająca kontury to wektor wektora punktów, ponieważ pojedyńcza kontura, tj. obiekt zawierający obrys interesującego nas obiektu jest zbiorem punktów, czyli wektorem. Natomiast funkcja drawContours rysuje zbiór kontur, bo w danej klatce może być więcej niż jeden osobny obiekt. Zatem musimy jej przekazać zbiór obrysów interesujących nas obiektów, czyli wektor wektorów punktów.

4. Oprócz narysowania konturu na obszarze zainteresowań, narysuj również prostokąty otaczające obiekt na oryginalnym obrazie:

   ```
   cv::Rect rect = cv::boundingRect(contours[i]);
   cv::rectangle(frame_roi, rect, cv::Scalar(0, 255, 0), 3);
   ```

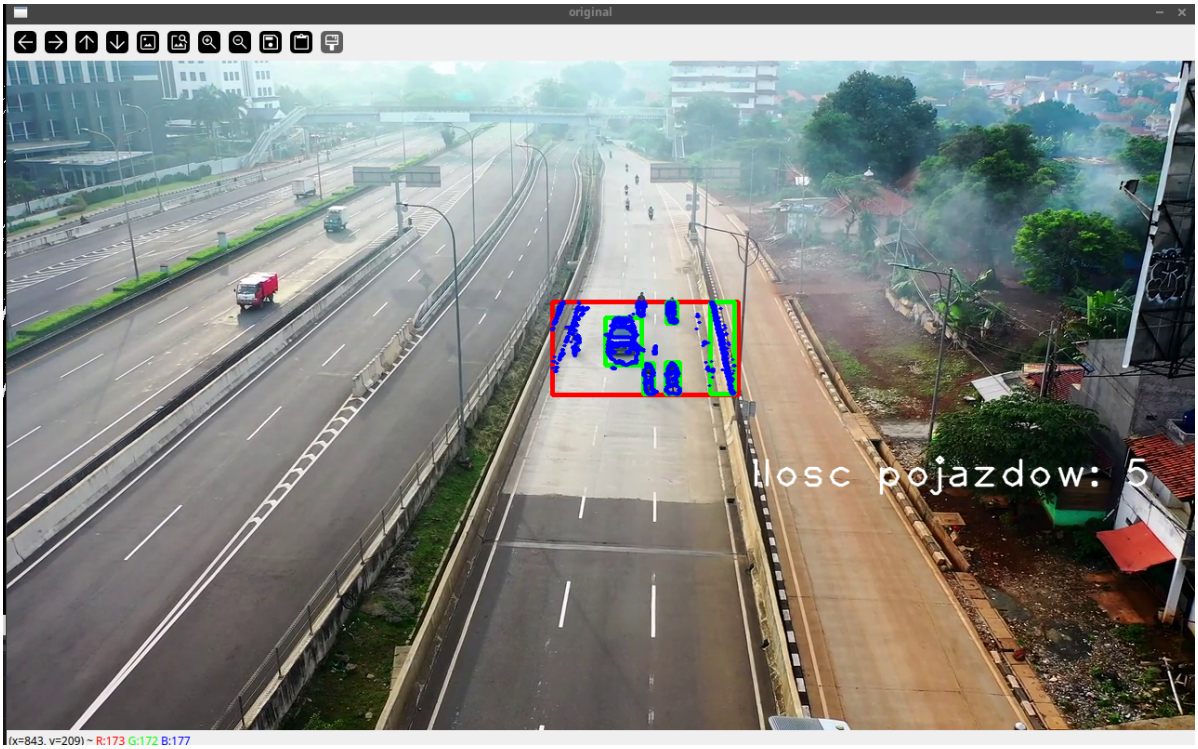
   Dodatkowo narysuj kwadrat na oryginalnej klatce otaczający cały wyodrębiony obszar.



Rysunek 11: Narysowane kontury obiektów, ramka na obiektach oraz ramka obszaru zainteresowań

5. Narysuj na ekranie tekst wskazujący ile obecnie wykrytych jest pojazdów. Przed pętlą `for` stwórz zmienną, która zostanie inkrementowana przy spełnieniu warunku `area > 100`. Po wykonaniu tej pętli wyświetl tekst za pomocą:

```
cv::putText(frame_In, "Ilosc pojazdow: " + std::to_string(car_number),
↪  cv::Point(x, y), cv::FONT_HERSHEY_PLAIN, 3, cv::Scalar(255, 255, 255), 2,
↪  cv::LINE_AA);
```



Rysunek 12: Zliczanie pojazdów

Ćwiczenie 3. Operacje morfologiczne

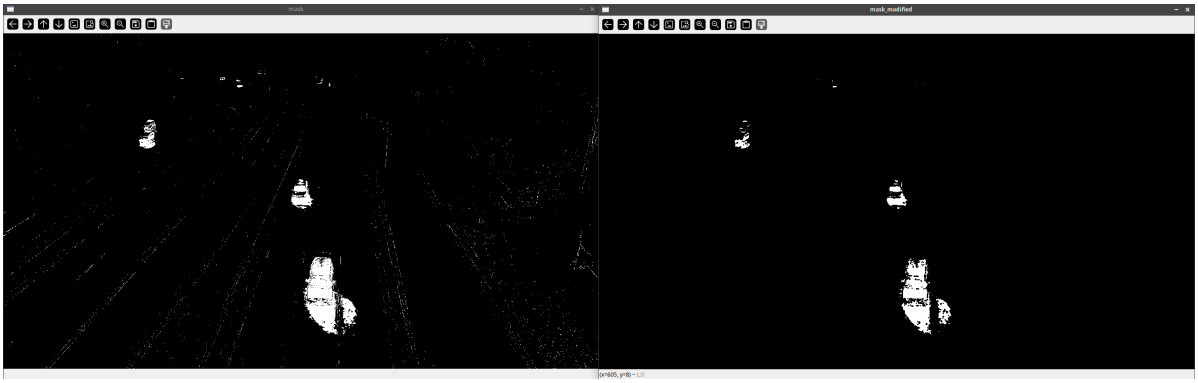
Uzyskana maska posiada wiele artefaktów. Można zmniejszyć ich wpływ wykonując późniejsze operacje na obrazie. W tym celu:

1. Upewnij się, że wyłączone jest wykrywanie cieni (MOG2 zwraca tylko kolor biały lub czarny). Przetestuj różny wpływ operacji morfologicznych z jednakowym elementem (jądrem):

   ```
   cv::Mat kernel = cv::getStructuringElement(cv::MORPH_ELLIPSE, cv::Size(3, 3));
   cv::morphologyEx(frame_in, frame_out, cv::MORPH_<wybrana_operacja>, kernel);
   // Przykład: cv::MORPH_ERODE
   ```

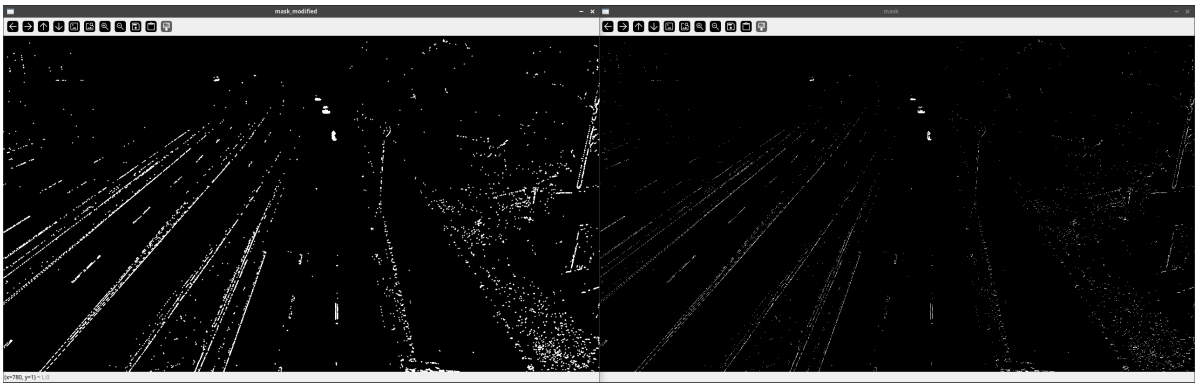
   Wykonaj tą operację na całej masce i wykonaj zdjęcia zawierające zmodyfikowaną maskę oraz obraz oryginalny z naniesionymi konturami. Sprawdź m.in.:

a) erozję,



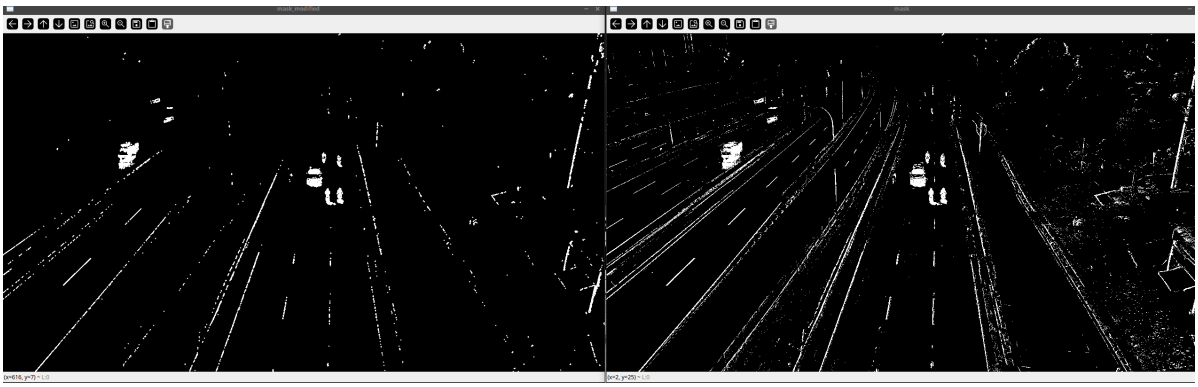
Rysunek 13: Erozja

b) dylatację,



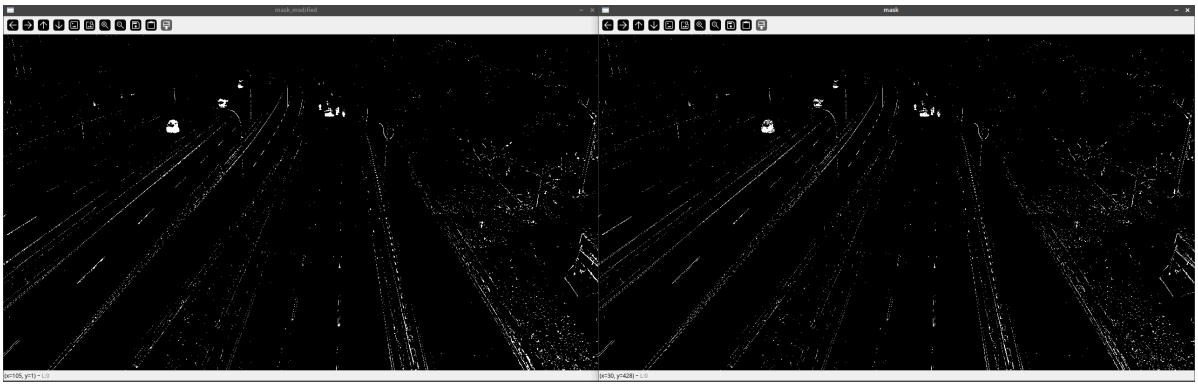
Rysunek 14: Dylatacja

c) otwarcie



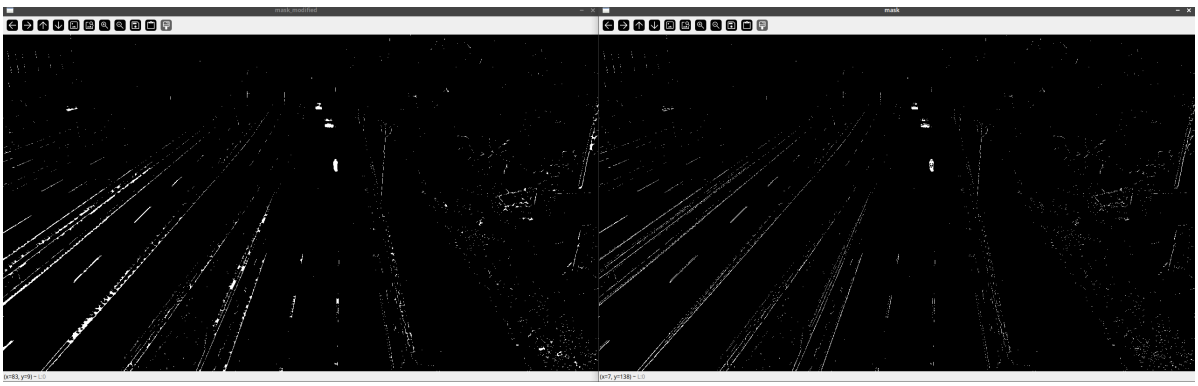
Rysunek 15: Otwarcie

d) zamknięcie,



Rysunek 16: Zamknięcie

e) otwarcie + zamknięcie,



Rysunek 17: Otwarcie + zamknięcie

f) zamknięcie + otwarcie.



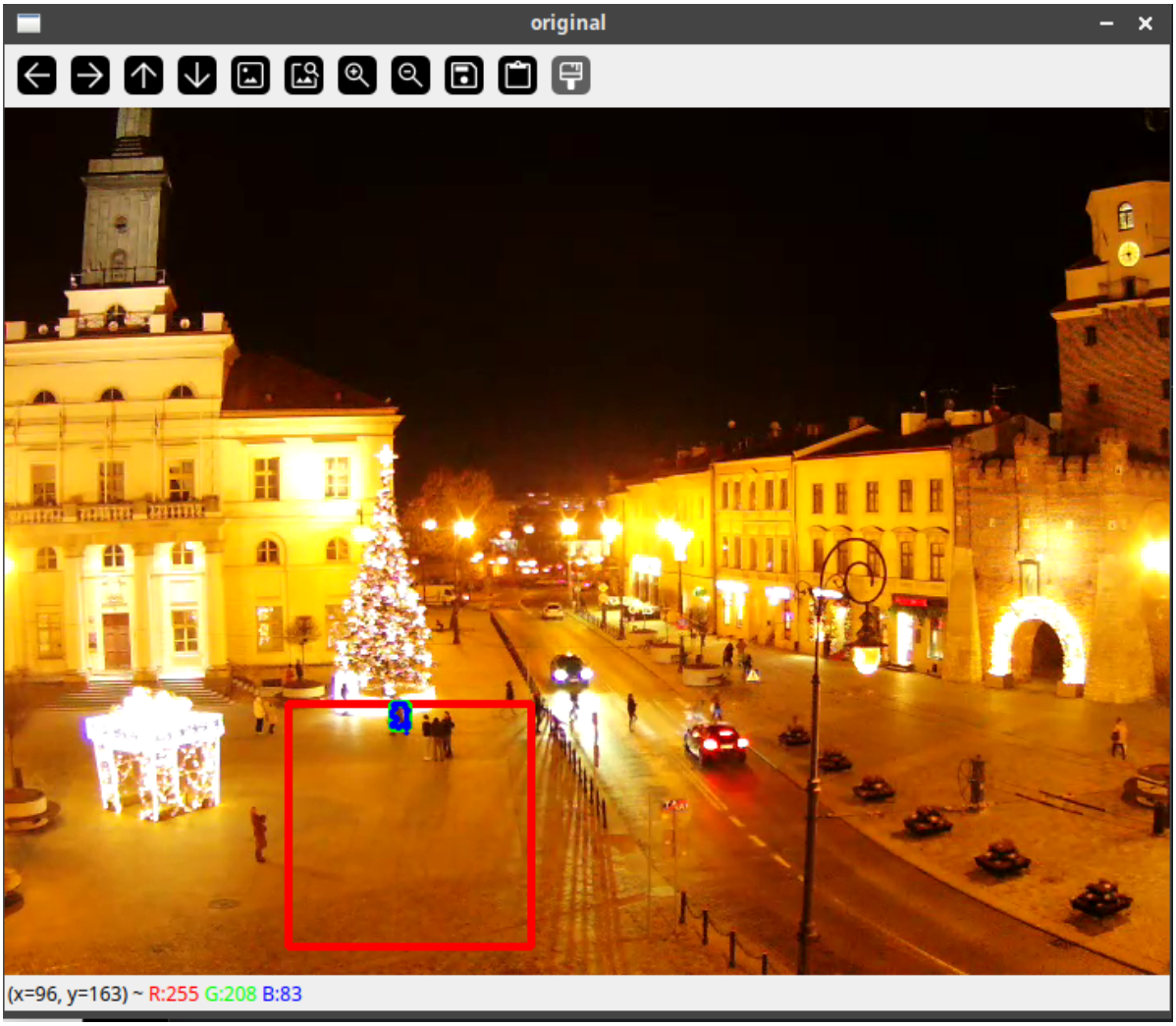
Rysunek 18: Zamknięcie + otwarcie

Ich listę można znaleźć przechodząc do definicji tych typów (CTRL + LPM na jeden z nich). Która z tych operacji lub ich kombinacji najlepiej pozwala zidentyfikować pojazdy?

Moim zdaniem najlepszy jest zestaw operacji otwarcie oraz zamknięcie (elipsa 2x2).

Ćwiczenie 4. Zliczanie ruchomych obiektów

Sprawdź utworzony algorytm na innym źródle, np. na bramie krakowskiej. Spróbuj wyświetlać ile samochodów jest obecnie w wybranym miejscu drogi, a ile osób znajduje się przy samym wejściu do bramy. Skorzystać z widoku od strony ul. Królewskiej.



Rysunek 19: Algorytm na innym źródle

```
#include <opencv2/objdetect.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/videoio.hpp>
#include <opencv2/video/background_segm.hpp>
#include <opencv2/bgsegm.hpp>
#include <iostream>
#include <iomanip>

int main(int argc, char **argv) {
```

```
cv::Mat frame_ref, frame_mask, frame_mask_mod;

//auto capture = cv::VideoCapture("highway.mp4");
auto capture = cv::VideoCapture("https://1510162109.rsc.cdn77.org/liv
auto subtractor = cv::createBackgroundSubtractorMOG2(100, 100, false)

int roi_x = 175;
int roi_y = 368;

int roi_width = 150;
int roi_height = 150;

while (capture.read(frame_ref)) {
        if (cv::waitKey(40) & 0xFF == 27) {
                break;
        }

        subtractor->apply(frame_ref, frame_mask);

        cv::Mat kernel = cv::getStructuringElement(cv::MORPH_ELLIPSE,
        cv::morphologyEx(frame_mask, frame_mask_mod, cv::MORPH_OPEN,
        cv::morphologyEx(frame_mask_mod, frame_mask_mod, cv::MORPH_CL
        // Przykład: cv::MORPH_ERODE

        cv::Mat frame_roi = frame_mask_mod(cv::Rect(roi_x, roi_y, roi



        std::vector<std::vector<cv::Point>> contours;
        std::vector<cv::Vec4i> hierarchy;
        cv::findContours(frame_roi, contours, hierarchy, cv::RETR_TRE
        cv::rectangle(frame_ref, cv::Point(roi_x, roi_y),cv::Point(ro

        int car_number=0;
```

```
                for (size_t i = 0; i < contours.size(); i++) {
                        double area = cv::contourArea(contours[i]);
                        if (area > 100) {
                                car_number++;
                                cv::Rect rect = cv::boundingRect(contours[i])
                                cv::rectangle(frame_ref, rect, cv::Scalar(0,
                                cv::drawContours(frame_ref, contours, -1, cv:
                                2, cv::LINE_AA, hierarchy, 3, cv::Point(roi_x
                        }
                }
                cv::putText(frame_ref, "Ilosc pojazdow: " + std::to_string(ca

                cv::imshow("original", frame_ref);
                cv::imshow("mask", frame_mask);
                cv::imshow("mask:Roi", frame_roi);
                cv::imshow("mask_modified", frame_mask_mod);
        }

        capture.release();
        cv::destroyAllWindows();
        return 0;
}
```

Zakończenie

Upewnij się, że strona tytułowa jest uzupełniona. Prześlij wygenerowany plik `.pdf` jako zadanie na platformę Moodle (aktualny plik `.pdf` tworzony jest w momemcie kompilacji w programie TexStudio). Zamknij plik w programie TexStudio i projekt w programie CLion (*File -> Close project*).